

LAPORAN TUGAS BESAR
ANALISIS KOMPLEKSITAS ALGORITMA

ANALISIS PERBANDINGAN KOMPLEKSITAS WAKTU
ASIMTOTIK ALGORITMA ITERATIF DAN REKURSIF
PADA OPERASI DOT PRODUCT



DISUSUN OLEH
ZAKY MUHAMMAD FAUZI (103052400064)
DS-48-03

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2025

BAB I: PENDAHULUAN

A. Latar Belakang

Mata Kuliah Analisis Kompleksitas Algoritma bertujuan untuk memberikan pemahaman mendalam terkait efisiensi sebuah solusi komputasi. Fokus utama dari mata kuliah ini mencakup kemampuan mahasiswa dalam menghitung kompleksitas waktu asimtotik untuk algoritma iteratif dan rekursif, serta menentukan ketepatan penggunaan algoritma tersebut berdasarkan persoalan tertentu.

Meskipun pemahaman teoretis diuji melalui tes tertulis, pengerjaan Tugas Besar ini menjadi krusial untuk melihat esensi nyata dari perhitungan kompleksitas yang telah dipelajari. Melalui praktik langsung, mahasiswa dapat mengamati efek perbedaan efisiensi antara dua pendekatan algoritma pada berbagai ukuran masukan, yang mana sangat penting dalam dunia nyata, terutama di bidang Data Science yang sering kali menangani data berskala besar.

B. Tujuan

Berdasarkan latar belakang tersebut, pengerjaan tugas besar ini bertujuan untuk:

1. Mengasah kemampuan mahasiswa dalam membandingkan dua versi algoritma (iteratif dan rekursif) dari sisi efisiensi waktu eksekusi (running time).
2. Memahami pentingnya efisiensi algoritma dalam penyelesaian masalah komputasi nyata.
3. Mengidentifikasi batasan-batasan teknis (seperti penggunaan memori stack) pada algoritma rekursif dibandingkan dengan iteratif.

C. Deskripsi Studi Kasus Permasalahan

Dalam bidang Data Science dan Machine Learning, operasi vektor merupakan fondasi utama dari hampir seluruh proses komputasi data. Salah satu operasi yang paling fundamental adalah Dot Product (perkalian titik) antara dua vektor.

Operasi ini merupakan inti dari berbagai algoritma populer, seperti Linear Regression, Logistic Regression, dan terutama Neural Networks. Dalam Neural Networks, hasil aktivasi sebuah neuron didapatkan dengan melakukan operasi dot product antara vektor masukan (input) dan vektor bobot (weight). Karena model pembelajaran mendalam modern sering kali melibatkan jutaan hingga miliaran parameter, efisiensi dalam melakukan operasi dasar dot product ini menjadi sangat vital. Ketidakefisienan pada tingkat ini akan menyebabkan peningkatan waktu pelatihan model secara signifikan. Studi kasus ini membandingkan efisiensi pendekatan iteratif dan rekursif untuk melihat skalabilitasnya terhadap peningkatan dimensi data.

BAB II: DESKRIPSI ALGORITMA

A. Operasi Dot Product

$$\begin{aligned} \begin{bmatrix} a & b \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} &= [ax + by] \\ \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} &= \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix} \\ \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} w & x \\ y & z \end{bmatrix} &= \begin{bmatrix} aw + by & ax + bz \\ cw + dy & cx + dz \end{bmatrix} \end{aligned}$$

Gambar 1 Dot product

Dot Product (Perkalian Titik) adalah operasi matematika fundamental yang menghasilkan nilai skalar dari dua vektor. Operasi ini sangat penting dalam berbagai bidang komputasi, terutama dalam Machine Learning dan Neural Networks.

Perkalian titik (dot product) dari dua vektor A dan B berdimensi n didefinisikan sebagai jumlah dari perkalian elemen-elemen yang bersesuaian:

$$A \cdot B = \sum_{i=0}^{n-1} A_i \times B_i$$

Contoh sederhana:

Vektor A: [1, 2, 3]

Vektor B: [4, 5, 6]

Dot product:

$$\begin{aligned} &= (1 \times 4) + (2 \times 5) + (3 \times 6) \\ &= 4 + 10 + 18 \\ &= 32 \end{aligned}$$

B. Algoritma Dot Product Iteratif

Algoritma iteratif menyelesaikan perhitungan ini dengan menggunakan struktur perulangan tunggal (loop).

- Mekanisme: Menginisialisasi variabel total ke nol, kemudian melakukan perulangan dari indeks 0 hingga n-1. Pada setiap tahap, elemen pada indeks yang sama dikalikan dan ditambahkan ke total.
- Kompleksitas Waktu: Algoritma ini memproses setiap elemen tepat satu kali, sehingga memiliki kompleksitas waktu $O(n)$.

- Kompleksitas Ruang: Hanya memerlukan satu variabel tambahan untuk menyimpan hasil, sehingga kompleksitas ruangnya adalah $O(1)$.

```
def dot_product_iterative(vector_a, vector_b):  
    result = 0  
    n = len(vector_a)  
    for i in range(n):  
        result += vector_a[i] * vector_b[i]  
    return result
```

C. Algoritma Dot Product Rekursif

Algoritma rekursif memecah masalah besar menjadi sub-masalah yang lebih kecil menggunakan prinsip rekurensi.

- Mekanisme: Mengalikan elemen terakhir vektor dan menjumlahkannya dengan hasil pemanggilan fungsi diri sendiri untuk sisa vektor sepanjang $n-1$. Proses berhenti saat mencapai base case (ketika panjang vektor adalah nol).
- Kompleksitas Waktu: Terdapat n pemanggilan fungsi rekursif, sehingga kompleksitas waktunya secara teoretis tetap $O(n)$.
- Kompleksitas Ruang: Karena setiap pemanggilan fungsi menambah beban pada stack memori hingga mencapai base case, kompleksitas ruangnya adalah $O(n)$. Hal ini memicu risiko Stack Overflow pada input yang sangat besar.

```
def dot_product_recursive(vector_a, vector_b, n):  
    if n == 0:  
        return 0  
    return (vector_a[n-1] * vector_b[n-1]) + dot_product_recursive(..., n-1)
```

BAB II: ANALISIS DAN PEMBAHASAN

A. Implementasi Sistem dan Skenario Pengujian

Algoritma diimplementasikan menggunakan bahasa pemrograman Python dengan antarmuka berbasis web menggunakan library Streamlit. Aplikasi ini memungkinkan pengguna untuk menentukan ukuran vektor n dan secara otomatis menghasilkan data acak untuk pengujian.

Pengujian dilakukan untuk membandingkan performa kedua algoritma dengan rincian sebagai berikut:

- Ukuran Masukan (n): 10, 50, 100, 250, 500, 750, 1000, 1500, 2000, 2500, dan 3000.
- Metode Pengukuran: Setiap ukuran masukan dijalankan sebanyak 10 kali, kemudian diambil nilai rata-ratanya untuk memastikan data yang stabil dan akurat.

B. Hasil dan Analisis Perbandingan

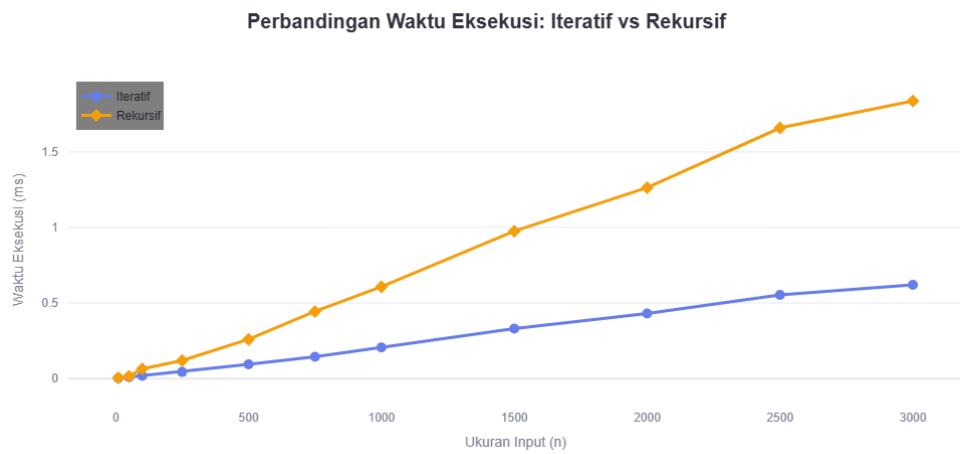
1. Hasil Pengujian

Berikut adalah tabel lengkap hasil benchmark rata-rata waktu eksekusi dalam milidetik (ms):

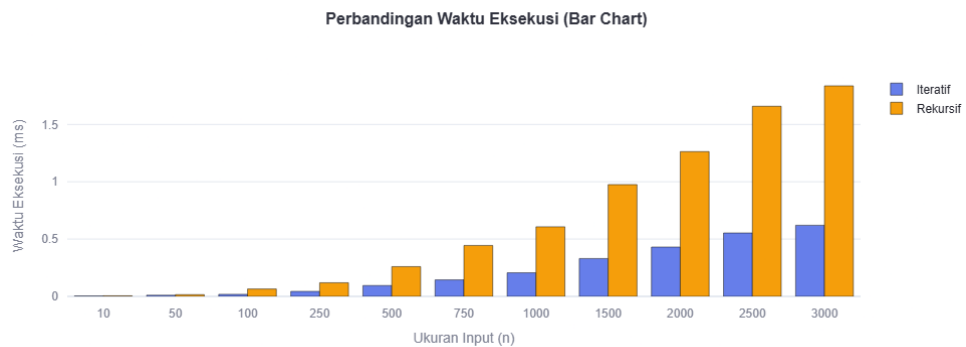
Ukuran (n)	Iteratif (ms)	Rekursif (ms)
10	0.0029	0.0048
50	0.0104	0.0154
100	0.0180	0.0649
250	0.0439	0.1192
500	0.0948	0.2599
750	0.1443	0.4447
1000	0.2062	0.6075
1500	0.3304	0.9755
2000	0.4307	1.2633
2500	0.5535	1.6590
3000	0.6207	1.8372

Tabel 1 Data hasil pengujian

Berikut adalah grafik perbandingan running time kedua metode:



Gambar 2 Tren kenaikan waktu seiring bertambahnya nilai n



Gambar 3 Perbandingan speedup antara kedua metode

2. Pembahasan

a. Algoritma Iteratif

```
def dot_product_iterative(vector_a, vector_b):
    result = 0          # O(1)
    n = len(vector_a)   # O(1)
    for i in range(n):  # Loop n kali
        result += vector_a[i] * vector_b[i] # O(1)
    return result        # O(1)
```

Time Complexity:

Perhitungan:

- Inisialisasi: $O(1)$
- Loop: $n \times O(1) = O(n)$
- Return: $O(1)$

$$T(n) = O(1) + O(n) + O(1) = O(n)$$

$$T(n) = O(n)$$

Space Complexity:

Hanya menggunakan variabel konstan (result, n, i) tanpa alokasi memori tambahan yang bergantung pada ukuran input.

b. Algoritma Rekursif

```
def dot_product_recursive(vector_a, vector_b, n):
    if n == 0:          # O(1)
        return 0
    return (vector_a[n-1] * vector_b[n-1]) + # O(1)
        dot_product_recursive(..., n-1) # T(n-1)
```

Time Complexity:

Relasi Rekurensi:

$$T(n) = T(n-1) + O(1) \quad T(0) = O(1)$$

Penyelesaian:

$$T(n) = T(n-1) + c \quad T(n) = T(n-2) + 2c \quad \dots \quad T(n) = T(0) + nc = O(n)$$

$$T(n) = O(n)$$

Space Complexity:

Setiap pemanggilan rekursif menambah frame baru ke call stack. Dengan kedalaman rekursi n , dibutuhkan ruang $O(n)$.

c. Perbandingan Kompleksitas

Aspek	Iteratif	Rekursif
Kompleksitas Waktu (Time)	$O(n)$	$O(n)$
Kompleksitas Ruang (Space)	$O(1)$	$O(n)$
Overhead per Operasi	Rendah	Tinggi (function call overhead)
Risiko Stack Overflow	Tidak Ada	Ada (tergantung depth)
Skalabilitas	Sangat Baik	Terbatas

Kemudahan Implementasi	Mudah	Mudah
------------------------	-------	-------

Tabel 2 Perbandingan kompleksitas

d. Analisis Perbandingan

- Analisis Kecepatan: Secara keseluruhan, algoritma iteratif bekerja 2.73x lebih cepat dibandingkan algoritma rekursif. Perbedaan ini mulai terlihat signifikan saat $n \geq 100$.
- Overhead Fungsi: Meskipun keduanya memiliki kompleksitas $O(n)$, algoritma rekursif mengalami perlambatan karena adanya overhead pemanggilan fungsi. Setiap pemanggilan rekursif memerlukan waktu tambahan untuk alokasi memori pada stack, yang tidak terjadi pada perulangan iteratif sederhana.
- Efisiensi Memori: Algoritma iteratif jauh lebih efisien karena penggunaan memori yang konstan. Sedangkan pada algoritma rekursif, pengujian hingga $n=3000$ adalah batas yang cukup berisiko tanpa optimasi khusus, karena penggunaan memori stack berbanding lurus dengan jumlah data.
- Kesimpulan Analisis: Untuk kebutuhan pemrosesan data masif di bidang Data Science, pendekatan iteratif adalah pilihan yang paling tepat dari sisi efisiensi waktu dan penggunaan sumber daya.

C. Kesimpulan dan Saran

1. Kesimpulan

- a. Algoritma iteratif dan rekursif untuk dot product memiliki kelas kompleksitas waktu asimtotik yang sama, yaitu $O(n)$.
- b. Meskipun kedua algoritma memiliki kompleksitas waktu $O(n)$ yang sama, versi rekursif cenderung lebih lambat dalam praktiknya karena:
 - Function Call Overhead: Setiap pemanggilan rekursif membutuhkan waktu untuk menyimpan dan memulihkan konteks eksekusi (register, return address, dll).
 - Stack Memory Access: Akses ke call stack lebih lambat dibanding akses ke memori lokal dalam loop.
 - Cache Miss: Pemanggilan fungsi berulang dapat menyebabkan lebih banyak cache miss dibanding iterasi sequensial.
- c. Penggunaan rekursi pada data besar tetap tidak disarankan karena:
 - Risiko RecursionError atau crash
 - Penggunaan memori yang tinggi ($O(n)$ untuk call stack)
 - Performa yang lebih lambat dibanding iteratif
- d. Secara praktis, algoritma iteratif terbukti jauh lebih efisien (2.73x lebih cepat) dan lebih aman untuk data skala besar dibandingkan algoritma rekursif.

2. Saran

Untuk operasi Dot Product dan operasi vektor lainnya yang memproses data dalam jumlah besar, gunakan pendekatan iteratif atau manfaatkan library yang sudah dioptimasi seperti NumPy.

REFERENSI

- Anton, H., & Rorres, C. (2013). Elementary linear algebra: Applications version (1st ed.). Wiley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
- Nielsen, M. A. (2015). Neural networks and deep learning. Determination Press.
- Ummah, I., dkk. (2025). Buku ajar: Analisis kompleksitas algoritma. KBM.
- Levitin, A. (2011). Introduction to the design and analysis of algorithms (3rd ed.). Pearson.
- Neapolitan, R., & Naimipour, K. (2014). Foundations of algorithms (5th ed.). Jones & Bartlett Learning.
- Munir, R. (2006). Diktat strategi algoritmik IF2251. Departemen Teknik Informatika, Institut Teknologi Bandung.